

Engenharia de Contexto em RAG

Aula prática: do top-k ao contexto suficiente

Prof^a. Dra. Simone Dominico

Especialização em Inteligência Artificial Generativa Aplicada

Na aula anterior, o pipeline já fazia:

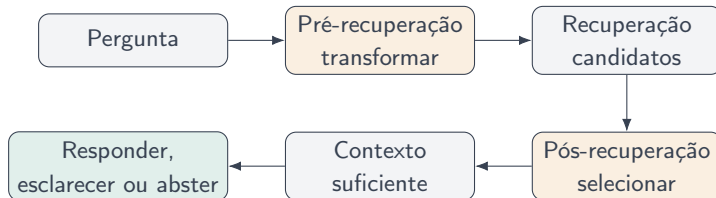
- leitura de PDFs;
- limpeza e segmentação em chunks;
- geração de embeddings com Sentence Transformers;
- indexação persistente no ChromaDB;
- busca top-k por similaridade;
- opção de MMR para reduzir redundância.

Objetivos da aula

Ao final da aula:

- explicar contexto como recurso escasso, disputado por instruções, histórico, ferramentas, evidências e resposta;
- medir custo aproximado e latência por etapa do pipeline;
- transformar consultas com rewrite, expansão, step-back e HyDE controlado;
- reconhecer riscos de viés quando a consulta é alterada;
- combinar múltiplas buscas e aplicar fusão RRF;
- aplicar filtros simbólicos, filtros semânticos, deduplicação e reranking;
- comprimir contexto por extração seletiva e discutir sumarização orientada à pergunta;
- decidir quando há contexto suficiente para responder, esclarecer ou abster-se;
- conectar RAG textual a fontes estruturadas e controladas de contexto.

Mapa conceitual



Diagnóstico

A aula não é apenas sobre recuperar mais chunks. O objetivo é controlar como a pergunta vira busca, como os candidatos são selecionados e quando existe evidência suficiente para responder.

Produto final da aula

Vamos avaliar, de forma incremental, uma função de alto nível:

```
responder_com_contexto(pergunta, escopo)
```

Ela executa:

1. análise da pergunta e possível ambiguidade;
2. transformação da consulta;
3. múltiplas recuperações e fusão;
4. filtros por metadado, score e escopo;
5. deduplicação textual e aproximada;
6. reranking com medição de latência;
7. compressão e normalização do contexto;
8. teste de contexto suficiente;
9. montagem do prompt ou decisão de esclarecimento/abstenção.

Perguntas de teste da aula

Usaremos as mesmas perguntas da aula anterior, porque elas provocam falhas diferentes:

- **O curso possui estágio obrigatório?**

Caso de evidência direta no PPC ou regulamento.

- **Qual é a frequência mínima?**

Caso ambíguo: pode ser disciplina, estágio ou regra geral do PPC.

- **A disciplina de Banco de Dados aborda SQL?**

Caso que exige filtro de escopo para não misturar documentos do curso.

Diagnóstico

A mesma infraestrutura pode responder bem uma pergunta e falhar em outra. Por isso, a aula trabalha com comparação de estratégias.

Na aula passada, nosso problema era encontrar chunks semelhantes.

Hoje vamos assumir que conseguimos encontrá-los e tratar um problema diferente: quais deles realmente merecem chegar ao LLM?

1. Partindo do código existente

Estrutura do código da aula anterior

```
from pathlib import Path
from time import perf_counter
import re

import chromadb
import numpy as np
from pypdf import PdfReader
from sentence_transformers import SentenceTransformer
```

- PdfReader: extrai texto dos PDFs.
- SentenceTransformer: gera embeddings.
- chromadb: armazena vetores, documentos e metadados.
- perf_counter: mede latência.
- numpy: usado para cálculo de MMR.

Configurações que viram decisões de projeto

```
TAMANHO_CHUNK = 800
OVERLAP_CHUNK = 150
TOP_K = 3
MODO_BUSCA = "similaridade" # ou "mmr"
FETCH_K_MMR = 12
LAMBDA_MMR = 0.6
RECRIAR_COLECAO = True
```

Decisão de projeto

Esses valores não são detalhes técnicos. Eles definem quanto contexto será criado, quanto será recuperado e quanto ruído pode entrar no prompt.

Como cada parâmetro afeta o comportamento

Parâmetro	Efeito prático
TAMANHO_CHUNK	Chunks grandes preservam contexto, mas carregam mais ruído.
OVERLAP_CHUNK	Overlap reduz corte de informação, mas aumenta redundância.
TOP_K	Controla quantos chunks entram como candidatos iniciais.
FETCH_K_MMR	Busca mais candidatos para permitir diversidade.
LAMBDA_MMR	Aproxima 1 favorece relevância; menor favorece diversidade.
RECRIAR_COLECAO	Define se a base será reconstruída ou reaproveitada.

O código da aula anterior já possui engenharia de recuperação.

- Ele lê documentos.
- Divide em unidades menores.
- Indexa embeddings.
- Mede latência.
- Recupera por similaridade.
- Permite MMR para reduzir redundância.

Agora falta engenharia do **contexto** que será enviado ao LLM.

2. Contexto como recurso escasso

Contexto não é apenas tamanho da janela

A janela de contexto é compartilhada por vários componentes.

Componente	Exemplo
Instruções	regras do sistema, formato da resposta, restrições
Histórico	mensagens anteriores e estado da interação
Ferramentas	descrição de funções e resultados de chamadas
Chunks RAG	evidências recuperadas dos documentos
Pergunta	solicitação atual do usuário
Resposta	espaço para o modelo produzir saída

Contexto é a memória de trabalho do modelo para uma execução.

Para a aula, vamos usar uma definição prática:

Context budget

Quantidade de tokens que podemos usar para instruções, pergunta, evidências, metadados e espaço de resposta sem degradar custo, latência ou qualidade.

Diagnóstico

O problema não é apenas “caber”. O problema é decidir o que merece ocupar espaço.

Exemplo de competição por tokens

Parte do prompt	Tokens aproximados
Instrução do sistema	600
Instruções da aplicação	500
Pergunta do usuário	80
Histórico necessário	1.200
5 chunks recuperados	3.500
Metadados e fontes	400
Espaço esperado para resposta	1.000
Total	7.280

Atividade prática

Se o limite fosse 6k tokens, o que você reduziria primeiro? Chunk? Histórico? Metadado? Resposta? Justifique pelo risco.

Contexto também afeta experiência do usuário

Decisão	Impacto no sistema	Impacto no usuário
Mais chunks	mais tokens e maior custo	resposta mais lenta e menos focada
Múltiplas buscas	mais chamadas ao índice	maior tempo até a resposta
Reranker caro	maior processamento	melhor ordenação, mas mais espera
Filtro agressivo	menor custo	risco de perder evidência
Compressão	menos tokens	resposta mais rápida, mas pode perder detalhe
Abstenção	menor cobertura	maior confiança no que é respondido

Decisão de projeto

A qualidade percebida pelo usuário depende de resposta correta, tempo aceitável e clareza sobre incerteza.

Código: estimar custo aproximado do contexto

```
import math

def estimar_tokens(texto: str) -> int:
    # Estimativa didática: 1 token ≈ 4 caracteres
    return math.ceil(len(texto) / 4)

def estimar_tokens_resultados(resultados: dict) -> int:
    documentos = resultados["documents"][0]
    return sum(estimar_tokens(doc) for doc in documentos)
```

A estimativa não substitui o tokenizer do modelo. Ela serve para tornar custo visível durante a prática.

Nós precisamos de uma medida aproximada para enxergar o custo das decisões que vamos tomar.

Mais contexto pode piorar

Quando aumentamos o top-k, podemos ganhar cobertura, mas também introduzir:

- regras de escopos diferentes;
- chunks quase duplicados;
- evidências fracas competindo com evidências fortes;
- trechos longos que escondem a informação principal;
- maior latência e maior custo.

O objetivo é enviar **contexto suficiente**, não contexto máximo.

Um chunk pode ser parecido com a pergunta e ainda não conter evidência suficiente.

Situação	Exemplo
Relacionado	fala de frequência, mas não informa percentual mínimo
Incompleto	menção estágio, mas não informa carga horária
Conflitante	mistura regra de disciplina com regra de estágio
Suficiente	contém valor, escopo, fonte e condição de aplicação

Diagnóstico

A meta da aula é sair de “chunks similares” para “contexto suficiente”.

Da busca ao contexto: três decisões diferentes

Etapas	Pergunta que responde	Risco principal
Recuperação	Quais chunks são parecidos com a consulta?	trazer conteúdo relacionado, mas insuficiente
Filtro	Quais chunks não deveriam entrar?	eliminar evidência útil por critério mal definido
Reranking	Em que ordem os chunks devem aparecer?	privilegiar sinal lexical e perder nuance semântica
Compressão	Que parte do chunk precisa ir ao prompt?	remover condição, exceção ou fonte
Suficiência	O contexto permite responder com segurança?	responder com evidência fraca

Decisão de projeto

Cada técnica modifica uma parte diferente do pipeline. Misturar essas funções dificulta diagnosticar por que a resposta final melhorou ou piorou.

3. Baseline: similaridade e MMR

Código existente: busca por similaridade

```
def buscar(collection, modelo, pergunta, k=TOP_K):  
    embedding_pergunta = modelo.encode(  
        pergunta,  
        normalize_embeddings=True,  
    ).tolist()  
  
    resultados = collection.query(  
        query_embeddings=[embedding_pergunta],  
        n_results=k,  
        include=["documents", "metadatas", "distances"],  
    )  
    return resultados
```

Diagnóstico

A busca mede proximidade vetorial. Ela não verifica se o chunk é suficiente para responder.

O que observar no baseline

Para cada pergunta, anotar:

Dimensão	Pergunta de diagnóstico
Top-1	O primeiro chunk contém a evidência principal?
Suficiência	O chunk permite responder sem inferência externa?
Escopo	A fonte é do curso, da disciplina ou do estágio?
Redundância	Há chunks muito parecidos no top-k?
Ruído	Há chunk relacionado, mas incapaz de responder?
Latência	Quanto tempo a recuperação levou?
Tokens	Quanto texto seria enviado ao LLM?

O problema da redundância na busca vetorial

A busca vetorial padrão recupera documentos de forma independente.

Se o corpus tiver vários fragmentos muito semelhantes, todos podem subir para o topo do ranking.

Consequências em produção:

- desperdício de tokens com a mesma informação repetida;
- janela de contexto menos diversa;
- evidências fortes competindo com cópias ou quase-cópias;
- maior risco de atenção difusa quando o contexto fica poluído.

O que é Maximal Marginal Relevance?

MMR é uma técnica de seleção e reordenação focada em diversidade.

A ideia central é selecionar o próximo chunk considerando dois fatores:

- relevância em relação à pergunta;
- redundância em relação aos chunks já escolhidos.

Diagnóstico

Em engenharia de contexto, MMR ajuda a usar melhor o orçamento de tokens: menos repetição, mais cobertura.

A fórmula do MMR

$$\text{MMR} = \arg \max_{d_i \in R \setminus S} \left[\lambda \cdot \text{Sim}_1(d_i, q) - (1 - \lambda) \cdot \max_{d_j \in S} \text{Sim}_2(d_i, d_j) \right]$$

Símbolo	Interpretação
q	consulta do usuário
R	candidatos recuperados inicialmente
S	documentos já selecionados
Sim_1	similaridade com a pergunta
Sim_2	similaridade com documentos já escolhidos
λ	equilíbrio entre relevância e diversidade

O parâmetro λ

Valor	Comportamento
$\lambda = 1.0$	prioriza relevância; aproxima o comportamento de top-k por similaridade
$\lambda = 0.0$	prioriza diversidade; pode ignorar demais a consulta original
$\lambda \approx 0.5\text{--}0.7$	tende a equilibrar relevância e diversidade em tarefas RAG

Decisão de projeto

Diminuir λ pode melhorar cobertura, mas também pode inserir ruído.

O algoritmo iterativo do MMR

1. Recuperar um conjunto maior de candidatos: `fetch_k`.
2. Selecionar primeiro o candidato mais relevante para a pergunta.
3. Para cada próximo candidato, calcular:
 - quanto ele se parece com a pergunta;
 - quanto ele se parece com o que já foi escolhido.
4. Escolher o candidato com maior relevância marginal.
5. Repetir até alcançar k resultados finais.

Benefícios

- reduz redundância;
- aumenta diversidade de evidências;
- melhora uso do orçamento de tokens;
- tem baixo custo quando aplicado sobre poucos candidatos.

Cuidados

- exige ajuste de λ ;
- diversidade excessiva pode trazer ruído;
- pode remover evidência repetida que ajudaria a confirmar a resposta;
- não resolve ambiguidade de escopo.

Código existente: MMR

```
# Ideia central do MMR no código:  
score_mmr = (  
    lambda_mult * relevancia_do_chunk  
    - (1 - lambda_mult) * maior_redundancia  
)
```

- A primeira parte preserva relevância para a pergunta.
- A segunda penaliza chunks parecidos com os já selecionados.
- O resultado pode reduzir redundância no contexto final.

Decisão de projeto

MMR pode trocar um chunk muito similar por outro mais diverso. Isso pode melhorar cobertura, mas também remover uma evidência redundante que ajudaria a confirmar a resposta.

Experimento 1: similaridade versus MMR

Atividade prática

No arquivo da aula anterior:

1. rode com `MODO_BUSCA = "similaridade"`;
2. faça as três perguntas de teste;
3. registre top-1, ruído, redundância e latência;
4. altere para `MODO_BUSCA = "mmr"`;
5. compare os resultados.

A pergunta principal: MMR melhorou a utilidade do contexto ou apenas deixou os resultados diferentes?

4. Pré-recuperação: transformar a consulta

Por que transformar a consulta?

A pergunta do usuário nem sempre é uma boa consulta de busca.

Problemas frequentes:

- pergunta curta demais;
- vocabulário diferente do documento;
- escopo implícito;
- ambiguidade;
- pergunta composta;
- falta de termos discriminativos.

Transformar consulta altera o espaço de busca

Pergunta original → Consulta transformada → Chunks recuperados

Ponto central

A transformação ocorre antes da recuperação. Portanto, ela decide quais regiões do espaço vetorial serão exploradas.

- Pode aumentar recall.
- Pode aproximar a pergunta da linguagem do documento.
- Pode inserir termos que não estavam na intenção original.
- Pode tornar uma suposição errada mais forte.

“Qual é a frequência mínima?”

Possíveis interpretações:

- frequência mínima para aprovação em uma disciplina;
- frequência ou assiduidade no estágio obrigatório;
- regra geral de frequência prevista no PPC do curso.

Diagnóstico

Se o sistema escolhe uma interpretação antes de resolver o escopo, ele pode recuperar evidência correta para a pergunta errada.

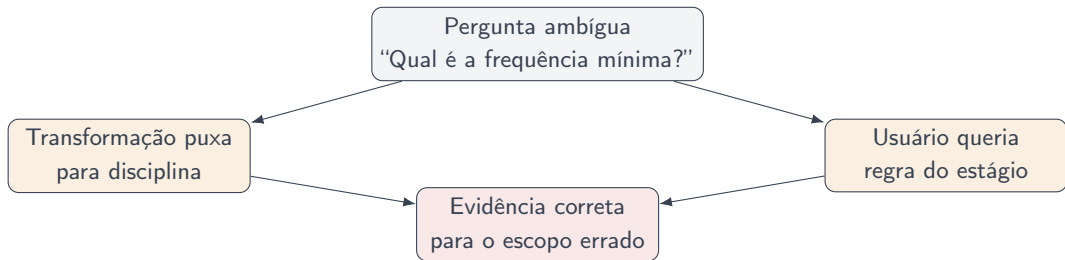
Pré-recuperação: antes de procurar, decidir como procurar

Técnica	O que muda	Quando ajuda
Rewrite	torna a pergunta mais explícita	quando a intenção está clara, mas a formulação é fraca
Expansão	adiciona sinônimos e termos institucionais	quando os documentos usam vocabulário diferente
Step-back	busca uma formulação mais geral	quando a pergunta específica depende de uma regra ampla
HyDE	gera um texto hipotético para buscar por semelhança	quando a pergunta é curta ou distante da linguagem documental

Diagnóstico

Essas técnicas não recuperam evidência. Elas alteram o caminho que a recuperação irá percorrer no espaço vetorial.

O risco central: a consulta transformada pode virar outra pergunta



Diagnóstico

Melhorar a recuperação não garante melhorar a resposta. Uma transformação pode aumentar recall e, ao mesmo tempo, deslocar a intenção do usuário.

Adicionar ao código: detecção simples de ambiguidade

Como vocês detectariam, de forma simples, que precisamos perguntar algo ao usuário antes de procurar?

Adicionar ao código: detecção simples de ambiguidade

Como vocês detectariam, de forma simples, que precisamos perguntar algo ao usuário antes de procurar?

```
def detectar_ambiguidade(pergunta: str) -> bool:
    p = normalizar_para_busca(pergunta)

    termos_ambiguos = [
        "frequencia minima",
        "carga horaria",
        "horas obrigatorias",
    ]

    return any(termo in p for termo in termos_ambiguos)
```

A regra é simples. O objetivo é mostrar que ambiguidade deve ser tratada como decisão do pipeline, antes da recuperação.

Adicionar ao código: query rewrite

Suponha que o usuário respondeu: disciplina. Como deveríamos procurar?

Adicionar ao código: query rewrite

Suponha que o usuário respondeu: disciplina. Como deveríamos procurar?

```
def reescrever_consulta(
    pergunta: str,
    escopo: str | None = None,
) -> str:
    p = normalizar_para_busca(pergunta)

    if "frequencia" in p:
        if escopo == "disciplina":
            return (
                "frequencia minima para aprovacao "
                "em disciplina regular"
            )

        if escopo == "estagio":
            return (
                "regra de frequencia ou assiduidade "
                "no estagio obrigatorio"
            )

        if escopo == "curso":
            return (
                "regra geral de frequencia minima "
                "prevista no PPC do curso"
            )

    return pergunta
```

Entrada	Consulta produzida
Pergunta original	Qual é a frequência mínima?
Escopo = disciplina	frequência mínima para aprovação em disciplina regular
Escopo = estágio	regra de frequência ou assiduidade no estágio obrigatório
Escopo = curso	regra geral de frequência prevista no PPC do curso

Diagnóstico

Resolver a ambiguidade não termina no rewrite: a decisão de escopo precisa ser preservada pelas transformações seguintes.

o documento não usar a expressão 'frequência mínima'?

Adicionar ao código: expansão de consulta

```
def expandir_consulta(
    pergunta: str,
    escopo: str | None = None,
) -> list[str]:
    p = normalizar_para_busca(pergunta)

    if "frequencia" in p:
        if escopo == "disciplina":
            return [
                pergunta,
                "presenca minima aprovacao disciplina",
                "limite de faltas disciplina regular",
            ]

        if escopo == "estagio":
            return [
                pergunta,
                "frequencia estagio obrigatorio",
                "assiduidade estagio supervisionado",
            ]

        if escopo == "curso":
            return [
                pergunta,
                "regra geral de frequencia PPC",
                "normas academicas de frequencia curso",
            ]

    return [pergunta]
```

Sem escopo definido, evitamos expansão agressiva: ela poderia ampliar justamente a interpretação errada.

Ganho	Risco
Encontra documentos com vocabulário diferente	Recupera documentos relacionados, mas não suficientes
Aumenta recall	Aumenta custo e latência
Melhora perguntas curtas	Multiplica fontes de ruído
Explora sinônimos e termos institucionais	Pode deslocar a intenção original

Se 'frequência mínima para aprovação em disciplina' é específica demais, qual seria uma consulta mais geral?

Adicionar ao código: step-back

```
def step_back(
    pergunta: str,
    escopo: str | None = None,
) -> str:
    p = normalizar_para_busca(pergunta)

    if "frequencia" in p:
        if escopo == "disciplina":
            return (
                "regras academicas de aprovacao "
                "e frequencia em disciplinas"
            )

        if escopo == "estagio":
            return (
                "regras de acompanhamento "
                "e assiduidade no estagio"
            )

        if escopo == "curso":
            return (
                "normas academicas gerais de "
                "frequencia previstas no PPC"
            )

    return pergunta
```

Step-back aumenta o nível de abstração sem abandonar o escopo já decidido.

Pergunta → documento hipotético → embedding → busca de documentos reais

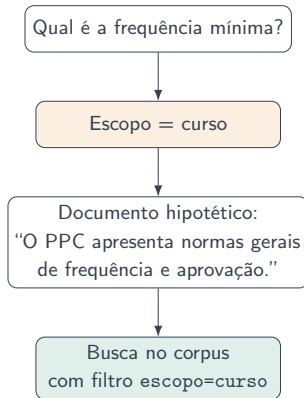
Adicionar ao código: HyDE controlado

```
def hyde_controlado(
    pergunta: str,
    escopo: str | None = None,
) -> str:
    p = normalizar_para_busca(pergunta)

    if "frequencia" in p:
        if escopo == "disciplina":
            return (
                "A ficha ou norma academica informa "
                "a frequencia minima para aprovacao."
            )
        if escopo == "estagio":
            return (
                "O regulamento de estagio estabelece "
                "regras de frequencia e assiduidade."
            )
        if escopo == "curso":
            return (
                "O PPC apresenta normas gerais "
                "de frequencia e aprovacao."
            )
    return pergunta

return pergunta
```

HyDE depois da resolução de escopo



Diagnóstico

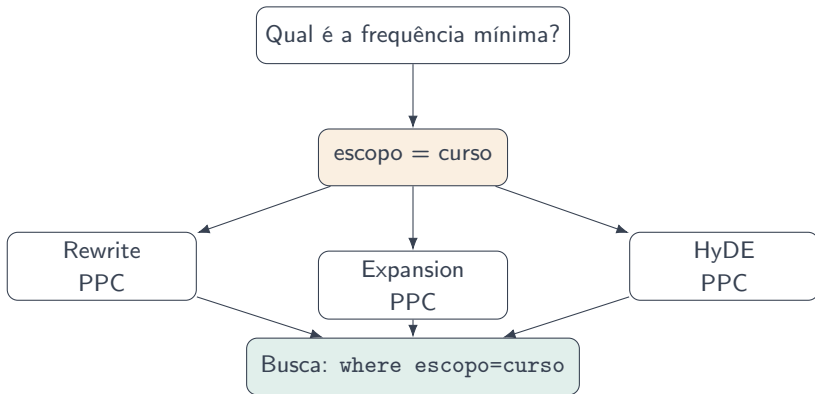
O documento hipotético é condicionado pelo escopo, mas continua não sendo evidência.

A decisão de escopo precisa sobreviver ao pipeline

O que acontece se rewrite usa curso, mas expansion usa termos de disciplina?

A decisão de escopo precisa sobreviver ao pipeline

O que acontece se rewrite usa curso, mas expansion usa termos de disciplina?



Diagnóstico

Uma transformação que ignora o escopo pode reintroduzir a ambiguidade já resolvida.

O documento hipotético ajuda a recuperar. Ele não é evidência.

Exemplo de falha:

- a pergunta é sobre estágio;
- o HyDE gera texto sobre disciplina;
- a busca encontra a ficha da disciplina;
- o LLM responde uma regra correta, mas para o escopo errado.

Diagnóstico

A transformação pode melhorar a métrica de recuperação e piorar a confiabilidade da resposta.

HyDE real versus simulado

	Na aula	Em produção
Geração	função determinística	LLM gera documento hipotético
Custo	baixo	chamada adicional ao modelo
Risco	viés codificado na regra	alucinação no documento hipotético
Uso correto	melhorar embedding de busca	buscar documentos, não gerar evidência

Decisão de projeto

O texto HyDE não deve ser tratado como fonte. Ele é uma isca semântica para localizar evidências reais.

Experimento 2: transformar a consulta

Atividade prática

Para a pergunta *“Qual é a frequência mínima?”*, compare:

1. pergunta original, sem escopo;
2. rewrite com escopo disciplina;
3. rewrite com escopo estagio;
4. rewrite com escopo curso;
5. expansão, step-back e HyDE respeitando o mesmo escopo.

Compare: evidência correta, escopo, ruído, latência e consultas geradas.

Agora temos várias consultas. O que fazemos com vários rankings?

5. Múltiplas buscas e fusão

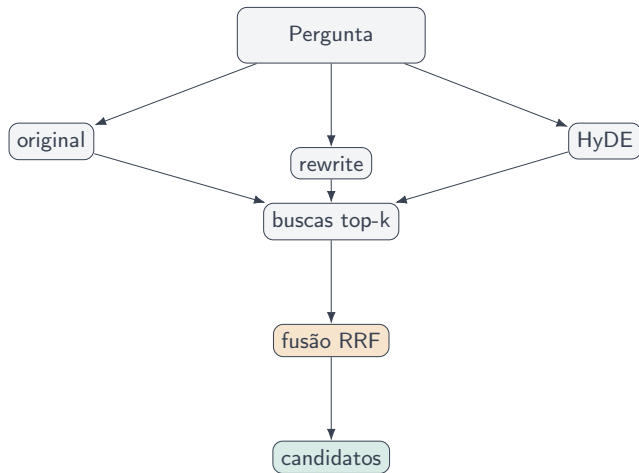
Por que múltiplas buscas?

Uma única consulta pode falhar por:

- usar vocabulário diferente dos documentos;
- representar apenas uma interpretação possível;
- recuperar chunks redundantes;
- não cobrir todos os aspectos da pergunta.

Múltiplas buscas aumentam robustez, mas também aumentam superfície de erro.

Pipeline com múltiplas buscas



Código: normalizar resultado do Chroma

```
from dataclasses import dataclass

@dataclass
class ResultadoChunk:
    id: str
    texto: str
    metadata: dict
    distancia: float
    similaridade: float
    consulta_origem: str

def converter_resultados(resultados: dict, consulta: str) -> list[ResultadoChunk]:
    itens = []
    for id_, doc, meta, dist in zip(
        resultados["ids"][0], resultados["documents"][0],
        resultados["metadatas"][0], resultados["distances"][0]
    ):
        itens.append(ResultadoChunk(id_, doc, meta, dist, 1 - dist, consulta))
    return itens
```

Código: buscar como lista de objetos

```
def buscar_lista(collection, modelo, pergunta: str, k: int = TOP_K):  
    resultados = buscar(collection, modelo, pergunta, k=k)  
    return converter_resultados(resultados, consulta=pergunta)
```

- O código original retorna o formato do Chroma.
- Para fusionar, filtrar e reordenar, é mais simples trabalhar com objetos.
- Essa função não muda a busca; apenas muda a representação dos resultados.

Se um documento aparece em 1º em três buscas diferentes, isso deveria contar?

Se um documento aparece em 1º em três buscas diferentes, isso deveria contar?

$$score(d) = \sum \frac{1}{k + pos(d)}$$

Intuição:

- chunk bem posicionado em várias consultas ganha força;
- chunk muito bom em uma consulta ainda pode sobreviver;
- evita comparar diretamente scores de consultas diferentes;
- simples o suficiente para usar em laboratório.

```
def fusao_rrf(listas: list[list[ResultadoChunk]], k_rrf: int = 60):  
    scores = {}  
    objetos = {}  
  
    for lista in listas:  
        for pos, item in enumerate(lista, start=1):  
            scores[item.id] = scores.get(item.id, 0.0) + 1.0 / (k_rrf + pos)  
            objetos[item.id] = item  
  
    ordenados = sorted(scores.items(), key=lambda x: x[1], reverse=True)  
    return [objetos[id_] for id_, _ in ordenados]
```

Código: multi-busca

```
def gerar_consultas(
    pergunta: str,
    escopo: str | None = None,
) -> list[str]:
    consultas = [
        pergunta,
        reescrever_consulta(pergunta, escopo),
    ]
    consultas.extend(expandir_consulta(pergunta, escopo))
    consultas.append(step_back(pergunta, escopo))
    consultas.append(hyde_controlado(pergunta, escopo))

    return list(dict.fromkeys(consultas))

def multi_busca(collection, modelo, pergunta, escopo=None, k=4):
    listas = []
    for q in gerar_consultas(pergunta, escopo):
        listas.append(
            buscar_lista(collection, modelo, q, k=k, escopo=escopo)
        )
    return fusao_rrf(listas)
```

6. Filtros, deduplicação e reranking

Agora que recuperamos mais, como vamos eliminar o que não queremos?

Pós-recuperação: reduzir candidatos sem perder evidência



Critérios semânticos e simbólicos

Critério	Exemplo	Força
Semântico	similaridade entre pergunta e chunk	encontra trechos relacionados mesmo com vocabulário diferente
Simbólico	escopo=curso, tipo=ppc	controla regras, fontes e domínios com precisão
Híbrido	similaridade dentro de um escopo filtrado	combina recall semântico com controle estrutural

Decisão de projeto

Quando o escopo é conhecido, o filtro simbólico deve entrar cedo para evitar que o espaço semântico misture regras de documentos diferentes.

A ordem das operações muda o resultado

Ordem	Efeito provável
Rerank antes dos filtros	pode promover chunks bem escritos, mas de escopo errado
Filtro antes do rerank	restringe o ranking aos candidatos admissíveis
Compressão antes do rerank	pode remover sinais úteis para ordenar
Deduplicação antes da compressão	evita gastar orçamento resumindo repetições

Diagnóstico

A ordem não é detalhe de implementação. Ela expressa a política de confiança do pipeline.

Separar as funções do pós-processamento

Etapa	Pergunta que responde
Filtro	Este chunk deve continuar candidato?
Deduplicação	Esta informação já apareceu?
Reranking	Qual chunk deve vir primeiro?
Compressão	Qual parte do chunk deve entrar no prompt?
Abstenção	Existe evidência suficiente para responder?

Misturar essas etapas dificulta diagnosticar falhas.

Filtro	Exemplo na aula
Escopo	manter apenas escopo = disciplina, estagio ou curso
Fonte	priorizar PPC, regulamento ou ficha oficial
Score	descartar similaridade baixa
Temporal	remover versão antiga do documento
Semântico	manter chunk que contém termos centrais da pergunta

Decisão de projeto

Filtro antes do reranking reduz custo. Filtro agressivo demais pode remover a única evidência correta.

Código: metadados de escopo

```
def inferir_escopo(arquivo: str, texto: str = "") -> str:
    alvo = normalizar_para_busca(f"{arquivo} {texto[:1000]}")
    if "estagio" in alvo:
        return "estagio"
    if "ci1218" in alvo or "disciplina" in alvo or "ficha" in alvo:
        return "disciplina"
    if "ppc" in alvo or "projeto pedagogico" in alvo or "curso" in alvo:
        return "curso"
    return "geral"
```

Diagnóstico

O escopo é uma informação simbólica. Ele não deve depender apenas da similaridade vetorial.

Código: indexar metadados controlados

```
metadatas=[{
    "arquivo": chunk["arquivo"],
    "pagina": chunk["pagina"],
    "numero_chunk": chunk["numero_chunk"],
    "escopo": chunk["escopo"],
    "tipo_documento": chunk["tipo_documento"],
    "fonte_controlada": True,
} for chunk in lote]
```

Esses metadados permitem filtros determinísticos antes do reranking e preparam a ponte para fontes estruturadas.

Filtro simbólico: antes e depois da recuperação

	Antes da busca	Depois da busca
Operação	<code>where escopo=curso</code>	verificar metadado dos candidatos
Efeito	reduz o espaço pesquisado	remove resultados indevidos
Custo	pode reduzir candidatos e processamento	opera apenas sobre o resultado
Risco	filtro errado elimina evidência antes da busca	pode descobrir o problema tarde

Decisão de projeto

Quando o escopo é conhecido e confiável, podemos incorporá-lo à própria recuperação.

Filtro dentro do banco vetorial

```
parametros = {
    "query_embeddings": [embedding_pergunta],
    "n_results": k_real,
    "include": [
        "documents",
        "metadatas",
        "distances",
    ],
}

if escopo is not None:
    parametros["where"] = {
        "escopo": escopo
    }

resultados = collection.query(**parametros)
```

O metadado simbólico restringe o espaço de busca antes do ranking por similaridade.

Filtro defensivo após a recuperação

```
def filtrar_por_escopo(candidatos, escopo):  
    if escopo is None:  
        return candidatos  
  
    return [  
        candidato  
        for candidato in candidatos  
        if candidato.metadata.get("escopo") == escopo  
    ]
```

Código: filtro por score

```
def filtrar_por_score(candidatos, minimo: float = 0.25):  
    filtrados = [c for c in candidatos if c.similaridade >= minimo]  
    return filtrados
```

Diagnóstico

Score baixo pode indicar baixa relação semântica, mas o valor absoluto depende do modelo, do corpus e da métrica. O limiar precisa ser calibrado.

Código: deduplicação textual

```
def hash_texto(texto: str) -> str:
    normalizado = normalizar_trecho_contexto(
        normalizar_para_busca(texto)
    )
    return hashlib.md5(normalizado.encode("utf-8")).hexdigest()[:12]

def deduplicar_textual(candidatos):
    vistos = set()
    saida = []
    for c in candidatos:
        h = hash_texto(c.texto)
        if h not in vistos:
            vistos.add(h)
            saida.append(c)
    return saida
```

Deduplicação: por que importa?

Chunks redundantes ocupam contexto e podem criar falsa confiança.

Exemplo:

- top-1: estágio obrigatório possui 220 horas;
- top-2: estágio obrigatório possui 220 horas;
- top-3: estágio obrigatório possui 220 horas.

O LLM recebe muita repetição e pouca cobertura de exceções, fonte, condição ou escopo.

Deduplicação textual versus aproximada

Tipo	Detecta	Pode falhar quando
Textual	trechos iguais ou quase iguais	a mesma regra aparece com palavras diferentes
Aproximada	trechos com vocabulário muito parecido	termos mudam muito ou há paráfrase forte
Semântica	equivalência de significado	exige embeddings ou modelo adicional

Decisão de projeto

Na aula, usamos deduplicação textual e aproximada. Em produção, MMR ou embeddings podem ajudar a reduzir redundância sem depender de igualdade textual.

Código: palavras relevantes

```
def palavras_relevantes(pergunta: str) -> set[str]:
    stopwords = {
        "qual", "quais", "como", "para", "possui",
        "sobre", "uma", "um", "de", "do", "da", "e", "o", "a",
    }
    termos = re.findall(r"\w+", pergunta.lower())
    return {t for t in termos if t not in stopwords and len(t) > 2}
```

Esse conjunto será usado no reranking e na compressão.

Código: reranking lexical

```
def rerank_lexical(pergunta: str, candidatos):
    termos = palavras_relevantes(pergunta)

    def score(c):
        texto = c.texto.lower()
        cobertura = sum(1 for t in termos if t in texto)
        bonus_fonte = 0.05 if c.metadata.get("arquivo") else 0.0
        return c.similaridade + 0.12 * cobertura + bonus_fonte

    return sorted(candidatos, key=score, reverse=True)
```

Diagnóstico

Esse reranker é simples. Ele é útil para aula porque torna visível o critério de reordenação.

Reranking: retriever versus reranker

	Retriever	Reranker
Objetivo	trazer candidatos	ordenar os candidatos
Prioridade	recall	precisão
Custo	menor	maior
Entrada	índice vetorial	lista de candidatos
Falha comum	ruído no top-k	viés no critério de reordenação

Decisão de projeto

Reranking não deve ser usado como desculpa para recuperar qualquer coisa. Ele é mais eficiente quando recebe candidatos razoáveis.

Medir custo do reranking

```
t = perf_counter()
candidatos = rerank_lexical(pergunta, candidatos)
tempos["reranking_ms"] = (perf_counter() - t) * 1000
```

Etapa	Ganha	Custa
Retrieval amplo	recall	mais candidatos e ruído
Reranking	melhor ordenação	processamento adicional
Compressão	menos tokens	risco de remover detalhe

7. Compressão do contexto

Compressão: reduzir tokens preservando capacidade de resposta

- Compressão não deve apenas encurtar texto.
- Ela deve preservar o que torna a resposta defensável.
- Em regras acadêmicas, valores, escopo, condições e fonte são críticos.
- Um contexto menor pode ser melhor se remover ruído e repetição.
- Um contexto menor pode ser pior se remover exceções ou critérios de aplicação.

Decisão de projeto

A pergunta não é “quanto posso cortar?”, mas “o que não posso perder para responder com segurança?”

O que a compressão precisa preservar

Elemento	Por que importa
Valor	horas, percentual, quantidade ou prazo usado na resposta
Escopo	define se a regra vale para curso, disciplina ou estágio
Condição	indica quando a regra se aplica
Exceção	evita generalização indevida
Fonte	permite rastrear de onde veio a evidência

Diagnóstico

Uma compressão que remove fonte ou condição pode melhorar tokens e piorar confiabilidade.

Mesmo depois de filtrar e reordenar, ainda há um problema:

O chunk pode ser relevante, mas grande demais.

Compressão pode significar:

- extrair apenas sentenças relevantes;
- remover repetição;
- normalizar espaços e quebras;
- preservar fonte, página e metadados;
- respeitar um limite de tokens.

A compressão deve preservar evidência, condição e fonte.

Riscos:

- remover exceções;
- juntar regras de documentos diferentes;
- transformar evidência fraca em afirmação forte;
- perder a página ou o arquivo de origem;
- induzir o LLM a responder sem base suficiente.

Compressão extrativa versus abstrativa

	Extrativa	Abstrativa
Como funciona	seleciona sentenças existentes	reescreve ou resume o trecho
Custo	baixo	requer chamada ao LLM
Risco	perder uma sentença importante	acrescentar informação não presente
Uso na aula	executado no código	mostrado como prompt de compressão

Diagnóstico

Compressão deve preservar evidência, fonte, números, condições e exceções.

Adicionar ao código: extração de sentenças úteis

```
def sentencas_relevantes(pergunta: str, texto: str, limite: int = 3):
    termos = palavras_relevantes(pergunta)
    sentencas = re.split(r"(?<=[.!?])\s+", texto.strip())

    selecionadas = [
        s for s in sentencas
        if any(t in s.lower() for t in termos)
    ]

    return selecionadas[:limite] or sentencas[:1]
```

Código: prompt para compressão abstrativa

```
def montar_prompt_compressao_abstrativa(pergunta, trecho):  
    return f"""  
Resuma o TRECHO apenas para responder à PERGUNTA.  
Preserve números, condições, exceções e fonte.  
Não acrescente informação externa.  
Se o trecho não ajudar, responda: IRRELEVANTE.  
  
PERGUNTA: {pergunta}  
TRECHO: {trecho}  
""".strip()
```

A aula executa a variante extrativa. Este prompt mostra como conectar uma etapa abstrativa quando houver uma API de LLM.

Código: comprimir contexto

```
def comprimir_contexto(pergunta: str, candidatos, limite_tokens: int = 700):
    contexto = []
    usados = 0

    for c in candidatos:
        trecho = " ".join(sentencas_relevantes(pergunta, c.texto))
        fonte = f"{c.metadata.get('arquivo')} p.{c.metadata.get('pagina')}"
        item = f"Fonte: {fonte}. Trecho: {trecho}"
        custo = estimar_tokens(item)

        if usados + custo <= limite_tokens:
            contexto.append(item)
            usados += custo

    return contexto, usados
```

Contexto bruto versus contexto comprimido

Critério	Chunks completos	Contexto comprimido
Tokens	maior	menor
Ruído	maior	menor, se bem comprimido
Risco	atenção difusa	perda de exceção
Fonte	pode estar em metadado	deve ser explicitada
Uso em aula	diagnóstico inicial	prompt final

Código: normalizar contexto

```
def normalizar_trecho_contexto(texto: str) -> str:
    texto = texto.replace("\\n", " ")
    texto = re.sub(r"\\s+", " ", texto)
    return texto.strip()
```

Normalização reduz ruído de formatação antes do prompt final, mas não deve alterar o sentido do trecho.

Código: montar prompt final

```
def montar_prompt(pergunta: str, contexto: list[str]) -> str:
    evidencias = "\n".join(f"[{i+1}] {c}" for i, c in enumerate(contexto))

    return f"""
Responda somente com base no CONTEXTO.
Se o contexto nao for suficiente, diga que nao ha evidencia suficiente.
Cite a fonte usada.

PERGUNTA:
{pergunta}

CONTEXTO:
{evidencias}
""".strip()
```

8. Decisão de resposta

Contexto suficiente é uma decisão, não uma métrica única

Pergunta	Critério mínimo
Qual é a frequência mínima?	precisa de percentual ou regra explícita de frequência
Quantas horas tem o estágio?	precisa de valor numérico e escopo do estágio
A disciplina aborda SQL?	precisa de evidência da ficha ou ementa da disciplina
O curso possui TCC?	precisa de regra do PPC ou matriz curricular

Diagnóstico

Um chunk relacionado pode justificar nova busca, mas não necessariamente uma resposta final.

Ações possíveis quando o contexto não é suficiente

Ação	Quando usar	Efeito no produto
Esclarecer	pergunta ambígua e sem escopo	aumenta interação, reduz erro de interpretação
Buscar novamente	há indício de evidência, mas cobertura fraca	aumenta latência, pode recuperar prova melhor
Abster-se	contexto não contém evidência suficiente	reduz cobertura, aumenta confiança
Responder	há evidência, fonte e escopo claros	entrega resultado com menor risco

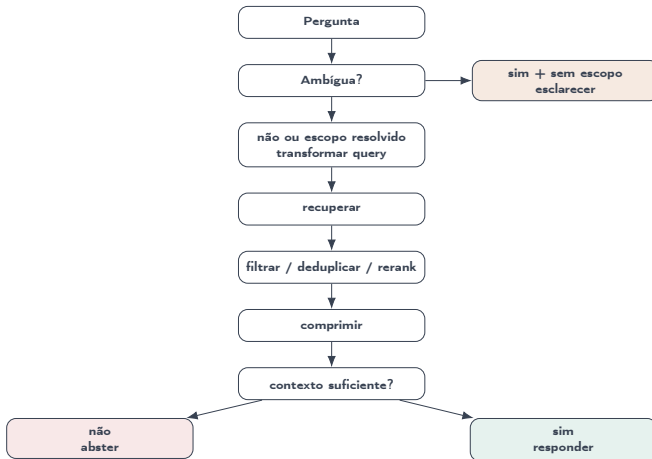
Responder nem sempre é a melhor ação

Um pipeline RAG confiável precisa escolher entre:

- responder com evidência;
- pedir esclarecimento;
- realizar nova busca;
- abster-se por falta de evidência.

Confiabilidade pode exigir reduzir cobertura.

Fluxo de decisão antes de chamar o LLM



Código: contexto suficiente

```
def contexto_suficiente(pergunta: str, contexto: list[str]):
    if not contexto:
        return False, "nenhum contexto foi selecionado"

    texto_contexto = " ".join(contexto)
    texto = normalizar_para_busca(texto_contexto)
    termos = palavras_relevantes(pergunta)
    cobertura = sum(1 for t in termos if t in texto)

    if cobertura == 0:
        return False, "termos centrais ausentes"

    p = normalizar_para_busca(pergunta)
    if any(t in p for t in [
        "quantas", "quanto", "carga horaria",
        "horas", "frequencia",
    ]):
        possui_valor = bool(re.search(r"\b\d+(?:[.],]\d+)?\b", texto_contexto)) or "%" in texto_contexto
        if not possui_valor:
            return False, "valor esperado ausente"

    return True, "há evidência mínima"
```

Estratégias que podem piorar uma métrica

Estratégia	Pode piorar	Pode melhorar
Pedir esclarecimento	tempo até resposta	precisão e escopo
Abster-se	cobertura	confiança
Filtrar agressivamente	recall	ruído e custo
Limitar top-k	cobertura	latência e foco
Comprimir contexto	completude textual	custo e legibilidade

Decisão de projeto

Em produto, nem sempre a melhor estratégia é a que responde mais vezes.

9. Pipeline final

Código final: decisão antes da busca

```
def responder_com_contexto(
    collection,
    modelo,
    pergunta,
    escopo=None,
):
    validar_escopo(escopo)

    if detectar_ambiguidade(pergunta) and escopo is None:
        return (
            "Acao: esclarecer\n"
            "Voce se refere a disciplina, estagio "
            "ou regra geral do curso?"
        )

    candidatos = multi_busca(
        collection,
        modelo,
        pergunta,
        escopo=escopo,
        k=4,
    )
```

Diagnóstico

Evitar uma recuperação desnecessária também é decisão de engenharia de contexto.

Código final: construir o contexto

```
candidatos = filtrar_por_escopo(candidatos, escopo)
candidatos = filtrar_por_score(candidatos, minimo=0.20)
candidatos = deduplicar(candidatos)
candidatos = rerank_lexical(pergunta, candidatos)

contexto, tokens = comprimir_contexto(
    pergunta,
    candidatos[:5],
    limite_tokens=700,
)

suficiente, motivo = contexto_suficiente(
    pergunta,
    contexto,
)

if not suficiente:
    return "Acao: abster\n" + motivo

return montar_prompt(pergunta, contexto)
```

Código final: medir etapas

```
tempos = {}

t = perf_counter()
candidatos = multi_busca(collection, modelo, pergunta, escopo=escopo, k=4)
tempos["multi_busca_ms"] = (perf_counter() - t) * 1000

t = perf_counter()
candidatos = rerank_lexical(pergunta, candidatos)
tempos["reranking_ms"] = (perf_counter() - t) * 1000

t = perf_counter()
contexto, tokens = comprimir_contexto(pergunta, candidatos[:5])
tempos["compressao_ms"] = (perf_counter() - t) * 1000
```

Diagnóstico

Sem medir etapa por etapa, não sabemos se a técnica melhorou o produto ou apenas deixou o sistema mais caro.

Experimento final: o mesmo texto, quatro decisões

```
testes = [  
    ("Qual e a frequencia minima?", None),  
    ("Qual e a frequencia minima?", "disciplina"),  
    ("Qual e a frequencia minima?", "estagio"),  
    ("Qual e a frequencia minima?", "curso"),  
]  
  
for pergunta, escopo in testes:  
    print("=" * 80)  
    print("Escopo:", escopo)  
  
    if escopo is not None:  
        for q in gerar_consultas(pergunta, escopo):  
            print(" ->", q)  
  
    print(responder_com_contexto(  
        collection,  
        modelo,  
        pergunta,  
        escopo,  
    ))
```

O que a solução final demonstra

- O mesmo corpus pode ser explorado por diferentes consultas.
- Aumentar recuperação melhora robustez, mas exige controle posterior.
- Filtro e reranking têm objetivos diferentes.
- Compressão reduz custo, mas precisa preservar evidência.
- Ambiguidade deve ser tratada antes da resposta.
- Um sistema RAG é uma cadeia de decisões, não apenas uma busca vetorial.

Checklist para diagnóstico de RAG

Falha observada	Onde investigar
Evidência não aparece	extração do PDF, chunking, embedding, top-k
Evidência aparece tarde	reranking, expansão, MMR
Muitos chunks repetidos	overlap, deduplicação, MMR
Escopos misturados	metadados, filtros, reescrita da consulta
Resposta sem base	prompt, compressão, regra de abstenção
Latência alta	top-k, número de consultas, reranker, tamanho do contexto

Ponte conceitual: nem todo contexto deve vir de RAG textual

Fonte	Exemplo	Vantagem
Texto recuperado	trecho do PPC ou regulamento	flexível, cobre conteúdo não estruturado
Metadados	escopo, tipo de documento, vigência	controla o espaço de busca
SQL	catálogo de disciplinas, carga horária, pré-requisitos	resposta determinística para dado estruturado
API	estado atual, calendário, serviço externo	contexto atualizado
Regras explícitas	política de frequência ou abstenção	comportamento previsível

Decisão de projeto

A engenharia de contexto decide qual fonte deve alimentar o modelo, e não apenas quantos chunks serão enviados.

Nem todo contexto precisa vir de busca vetorial.

Fonte de contexto	Quando usar
Documentos recuperados	evidência textual, normas, regulamentos
Metadados	escopo, fonte, data, versão, tipo de documento
SQL	fatos estruturados e verificáveis
APIs	estado externo atualizado
Regras controladas	validações, políticas, limites e restrições

Decisão de projeto

A próxima aula parte desta pergunta: quais partes do contexto devem ser recuperadas semanticamente e quais devem ser controladas de forma estruturada?

- Contexto é recurso escasso e compete com instruções, histórico e resposta.
- Top-k por similaridade é apenas o ponto de partida.
- Transformações de consulta alteram o espaço de busca e podem introduzir viés.
- Múltiplas buscas aumentam recall e complexidade.
- Filtros reduzem ruído; reranking muda ordem; compressão reduz conteúdo.
- Contexto suficiente combina evidência, escopo, fonte e condições.
- Confiabilidade pode exigir esclarecimento ou abstenção.

O melhor contexto é o menor contexto
suficiente.